

M13 — Como otros lo hicieron

FloripaSat, OreSat, AAUSAT, UPSat y NASA

Estudio comparativo de CubeSats open-source y frameworks de vuelo
Que copiar, que adaptar y que ignorar para el INTISAT

Equipo INTISAT

Mayo 2026

Prefacio

Este manual existe porque **no estamos solos**. Antes que nosotros, varios equipos universitarios pusieron CubeSats en orbita publicando *todo* su trabajo: codigo, esquematicos, documentos de diseno, lecciones aprendidas. Tambien la NASA libero dos de los frameworks de software de vuelo mas usados del mundo (cFS y F'). Ignorar todo eso seria reinventar problemas que ya fueron resueltos.

Para que sirve este manual. Es la respuesta sistematica a tres preguntas que aparecen una y otra vez en las reuniones:

1. “Esto que estamos haciendo, ¿como lo hace otra gente?”
2. “¿Hay un framework que ya resuelva esto y no tengamos que escribirlo nosotros?”
3. “Si la NASA usa esto en Marte, ¿por que no usamos lo mismo nosotros?”

Que no es este manual. No es un tutorial para usar cFS ni para escribir codigo F'. Es un *mapa* que muestra que existe, donde buscarlo y, sobre todo, *cuando vale la pena* usarlo y cuando no.

Como leerlo. Los capitulos 1 y 2 son el marco: timeline y matriz comparativa. Los capitulos 3 al 8 son uno por proyecto: leelos en cualquier orden, son independientes. El capitulo 9 cierra con la recomendacion concreta para INTISAT y la planilla de decisiones abiertas.

Índice general

1. Por que mirar lo que otros hicieron	1
1.1. La paradoja del CubeSat universitario	1
1.2. El movimiento open-source en CubeSats	1
1.3. Las 6 referencias que vamos a estudiar	2
2. Timeline y matriz comparativa	3
2.1. Linea de tiempo	3
2.2. Matriz comparativa	3
2.3. Como leer el resto del manual	4
3. FloripaSat (UFSC, Brasil)	6
3.1. El proyecto en 2 paginas	6
3.2. Arquitectura distribuida	6
3.3. Como manejan modos / FSM	7
3.4. Que adopta INTISAT y que no	8
3.5. Referencias	8
4. OreSat (Portland State University, USA)	9
4.1. El proyecto en 2 paginas	9
4.2. La arquitectura “cards” sobre bus CAN	9
4.3. Como manejan modos / FSM	10
4.4. Que adopta INTISAT y que no	11
4.5. Referencias	11
5. AAUSAT (Aalborg University, Dinamarca)	12
5.1. El proyecto en 2 paginas	12
5.2. Arquitectura monolitica + CSP	12
5.3. Como manejan modos / FSM (HSMM)	13
5.4. Que adopta INTISAT y que no	13
5.5. Referencias	14
6. UPSat (Libre Space Foundation, Grecia)	15
6.1. El proyecto en 2 paginas	15
6.2. Arquitectura	15
6.3. Como manejan modos / FSM	15
6.4. Que adopta INTISAT y que no	16
6.5. Referencias	16
7. NASA cFS (Core Flight System)	17
7.1. Que es cFS y por que importa	17
7.2. El stack en capas	17
7.3. Aplicaciones modulares: el patron clave	18

7.4. Que adopta INTISAT y que no	19
7.5. Referencias	19
8. NASA F'(FPrime)	20
8.1. Que es F'y por que importa	20
8.2. Componentes, ports y FPP	20
8.3. Que adopta INTISAT y que no	21
8.4. Referencias	22
9. Sintesis y recomendaciones para INTISAT	23
9.1. Lo que NO debe pasar	23
9.2. Recomendaciones de adopcion selectiva	23
9.3. Stack final propuesto para INTISAT	24
9.4. Planilla de decisiones abiertas	25
9.5. Cierre	25
A. Glosario de terminos	26
B. Indice de URLs verificables	28
B.1. Misiones universitarias	28
B.2. Frameworks NASA	28
B.3. Protocolos y estandares	28
B.4. Infraestructura terrestre open	28

Capítulo 1

Por que mirar lo que otros hicieron

1.1. La paradoja del CubeSat universitario

Un CubeSat universitario tipico vive una paradoja: por un lado, el equipo es chico, joven, sin experiencia previa en vuelo, con presupuesto limitado y plazos academicos. Por el otro, debe construir un sistema embebido distribuido, tolerante a fallas, con comunicaciones espaciales, calificacion ambiental, software certificable y operaciones 24/7 durante meses o anos. La unica forma honesta de cerrar esa brecha es *copiar inteligentemente*.

Intuicion

Copiar inteligentemente no es plagiar arquitectura completa. Es:

1. Identificar el problema que estas resolviendo.
2. Buscar quien lo resolvio antes en condiciones parecidas.
3. Estudiar su solucion y sus errores documentados.
4. Adoptar la idea, no necesariamente el codigo.

La diferencia con plagiar es que cuando copias inteligente, *entendes* lo que copias y puedes adaptarlo a tu contexto.

1.2. El movimiento open-source en CubeSats

Hasta aproximadamente 2014, la mayoría de los CubeSats eran “open” en nombre pero cerrados en la practica: publicaban papers academicos pero no el codigo ni los esquematicos. La situacion cambio cuando varios equipos universitarios empezaron a publicar TODO bajo licencias libres, y la NASA acompaño liberando sus frameworks internos. Las consecuencias son enormes:

- Un estudiante en cualquier universidad del mundo puede leer *exactamente* el codigo que vola en la ISS o en Marte.
- Los errores quedan documentados publicamente: si una mision fracasa, su post-mortem suele estar online.
- Los protocolos se estandarizaron: CSP (Cubesat Space Protocol), CANopen CiA 301, ECSS-E-ST-70-41 (telemetria/telecomando), aparecen una y otra vez en proyectos independientes.
- Las herramientas se compartieron: GNU Radio, gr-satnogs y la red SatNOGS de estaciones terrenas distribuidas son ejemplo de infraestructura comun.

1.3. Las 6 referencias que vamos a estudiar

Las elegimos por dos criterios: **(a)** estar 100 % publicas (codigo + docs) y **(b)** representar un punto distinto del espacio de diseno.

- **FloripaSat (UFSC, Brasil).** El ejemplo de America Latina. Arquitectura *distribuida*: 4 microcontroladores MSP430, cada uno con su propio firmware en FreeRTOS, comunicandose por SPI. Documentacion en LaTeX dentro del mismo repo.
- **OreSat (Portland State University, USA).** El ejemplo moderno: “cards” independientes sobre bus CAN con CANopen, OBC corriendo Linux con flight software en Python. 100 % open hardware y software.
- **AAUSAT (Aalborg University, Dinamarca).** El ejemplo metodologico: introdujeron el documento “Operations Concept” como artefacto formal, jerarquia de modos (HSMM) y crearon el Cubesat Space Protocol (CSP) que hoy usan decenas de misiones.
- **UPSat (Libre Space Foundation, Grecia, 2017).** El primer CubeSat 100 % open en orbita. Modesto en alcance pero fundacional en filosofia.
- **NASA cFS (Core Flight System).** Framework modular de flight software de NASA Goddard. Usado en mas de 20 misiones incluyendo MMS, GPM, Lunar IceCube y la nave Orion de Artemis.
- **NASA F’(FPrime).** Framework de NASA JPL. El que volo en MarCO (primeros CubeSats a Marte, 2018) y en el helicoptero Ingenuity (72 vuelos en Marte, 2021–2024).

Capítulo 2

Timeline y matriz comparativa

2.1. Línea de tiempo

Para tener un marco temporal claro, conviene ver cuando lanzo cada misión y cuando se libero cada framework.

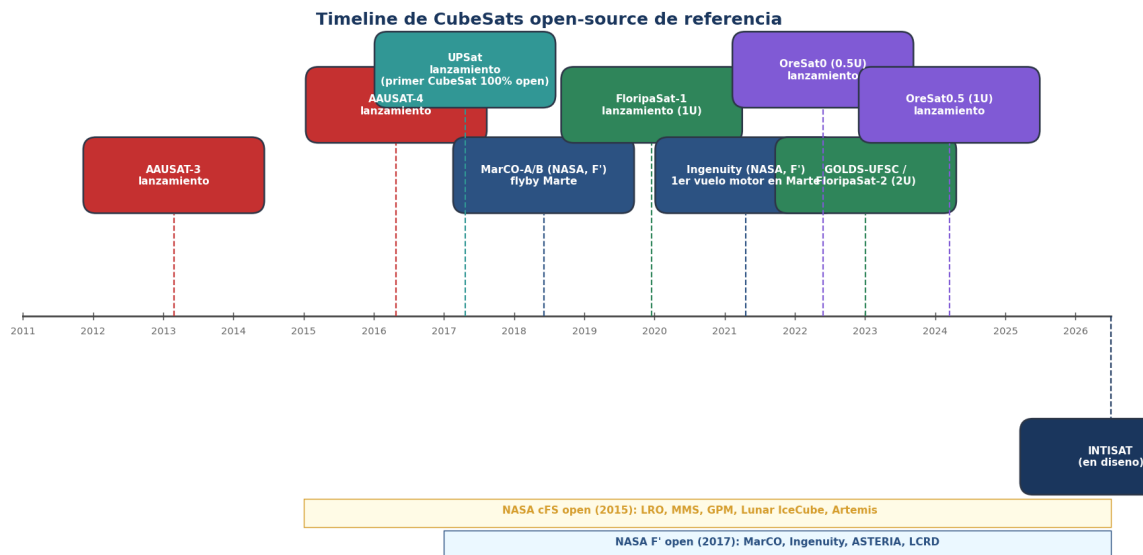


Figura 2.1: Timeline de las misiones y frameworks open-source de referencia. INTISAT al final, en diseño (2026).

Lectura del timeline. AAUSAT-3 (2013) fue la primera de las referencias académicas modernas en publicar TODO. UPSat (2017) marco el hito “100 % open en orbita”. Las misiones de NASA F empezaron a volar entre 2018 (MarCO) y 2024 (Ingenuity sigue activo), con NEA Scout, ASTERIA y LCRD entre medio. cFS, aunque liberado en 2015, ya estaba volando antes en versiones internas (LRO desde 2009).

2.2. Matriz comparativa

La siguiente matriz cruza los proyectos contra las 6 dimensiones que mas nos van a importar para tomar decisiones en INTISAT: arquitectura general, sistema operativo, bus interno, lenguaje, manejo de FSM y licencia.

Matriz comparativa: arquitecturas de CubeSat open-source

	Arquitectura	OS / runtime	Bus interno	Lenguaje	FSM	Open?
FloripaSat	Distribuida 4 módulos	FreeRTOS bare metal	SPI	C	FSM en C (plana)	Si
OreSat	Cards CANopen	Linux (Yocto)	CAN	Python	StateMachine explícita	Si
AAUSAT	Monolítica	FreeRTOS	I2C	C	HSMM (jerárquica)	Parcial
UPSat	Monolítica	FreeRTOS	I2C	C	Implícita	Si
NASA cFS	Componentes + pub/sub	VxWorks/RTMS Linux	Software Bus	C	Mode Manager app	Si
NASA F'	Componentes + ports	Linux VxWorks/RTMS	Ports FPP	C++	SMs generadas desde FPP	Si
INTISAT (target)	Monolítica payload-side	Linux (RPI 5)	I2C (con OBC)	Python	STATE_* en daemon.py	Si

INTISAT ya define payload-side; quedan abiertas las decisiones de OBC, bus inter-subsistema y framework

Figura 2.2: Matriz comparativa: arquitecturas y elecciones técnicas de cada proyecto. La fila amarilla destaca el target del INTISAT (lo ya decidido del lado del payload).

Intuición

Notar que NO hay una sola elección “correcta” en ninguna columna. *Todas* las combinaciones que aparecen en la tabla volaron y funcionaron. La pregunta no es “cual es la mejor” sino “cual es la mas conveniente para mi equipo, mi presupuesto, mi tiempo y mi misión”.

Patrones que vale la pena marcar:

- **C domina en RTOS.** FloripaSat, AAUSAT, UPSat y cFS estan escritos en C corriendo sobre FreeRTOS o RTEMS/VxWorks. Es la combinacion clasica de software de vuelo, con cinco decadas de pedigree.
- **Python aparece donde hay Linux.** OreSat (C3 card) y el target del INTISAT (Raspberry Pi 5) corren Python sobre Linux. Esto no era posible hace 10 años porque los OBC no tenían suficiente RAM. Hoy se puede.
- **Fúsa C++** (es la excepcion). Es la apuesta de JPL a un lenguaje con tipos mas fuertes y generacion automatica de codigo desde FPP.
- **Tres buses distintos aparecen: SPI, I2C y CAN.** SPI da el throughput mas alto pero requiere chip-select por esclavo. I2C es lo mas barato pero menos robusto. CAN es lo mas robusto pero requiere transceivers en cada nodo.

2.3. Como leer el resto del manual

Los proximos 6 capitulos siguen todos la misma estructura:

1. **El proyecto en 2 paginas.** Quien, donde, cuando.
2. **Arquitectura.** El diagrama de bloques del sistema.
3. **Como manejan modos / FSM.** Que aprendimos en M12.
4. **Que adopta INTISAT y que NO.** La decision concreta.
5. **Referencias.** URLs y papers verificables.

Si tenes poco tiempo, podes saltar a la seccion “Que adopta INTISAT” de cada capitulo y leer solo eso. Pero perderias el contexto.

Capítulo 3

FloripaSat (UFSC, Brasil)

3.1. El proyecto en 2 paginas

Que es. FloripaSat es un programa de CubeSats del SpaceLab del Departamento de Engenharia Eletronica e Eletrica de la Universidade Federal de Santa Catarina (UFSC) en Florianopolis, Brasil. Es la referencia mas cercana culturalmente al INTISAT: un equipo universitario de un pais latinoamericano que decidio publicar TODO y documentarlo en LaTeX.

Misiones.

- **FloripaSat-1** (1U, 1 kg). Lanzado el 20 de diciembre de 2019 desde el Centro de Lanzamiento de Taiyuan (China), en un cohete Long March 4B (CZ-4B) como secundario de la mision CBERS-04A. Sigue operativo a la fecha de este manual. Mision: demostracion tecnologica de subsistemas open-source y radio amateur.
- **FloripaSat-2 / GOLDS-UFSC** (2U). Lanzado en enero de 2023 en el SpaceX Transporter-6. Continuacion con payload cientifico de medicion de electrojet ecuatorial.

Filosofia. Cada subsistema del CubeSat es un repositorio de GitHub independiente, con su propio hardware, firmware y documentacion en LaTeX. La comunicacion entre subsistemas es por SPI. El equipo publico una serie de papers en conferencias LARS, IAA y SBC-WSAC con detalles tecnicos.

Lenguaje. Todo en C, sin ningun Python en orbita. El microcontrolador es MSP430 de Texas Instruments.

3.2. Arquitectura distribuida

Los 4 modulos son:

1. **OBDH (On-Board Data Handling).** El cerebro coordinador. MSP430 corriendo FreeRTOS. Mantiene el reloj, la FSM de modos operacionales, la telemetria global y la cola de telecomandos.
2. **EPS (Electrical Power System).** Otro MSP430, esta vez *bare metal* (sin RTOS) porque su tarea es muy acotada: leer ADCs de las celdas solares y baterias, controlar MPPT, apagar lineas de alimentacion cuando hay sobrecorriente.
3. **TT&C Beacon.** Transmisor automatico que cada cierto tiempo manda un paquete de telemetria basica en VHF para que estaciones de radio amateur de todo el mundo lo puedan recibir.
4. **TT&C Telecomando.** Receptor UHF que escucha comandos enviados desde la estacion terrena de UFSC.

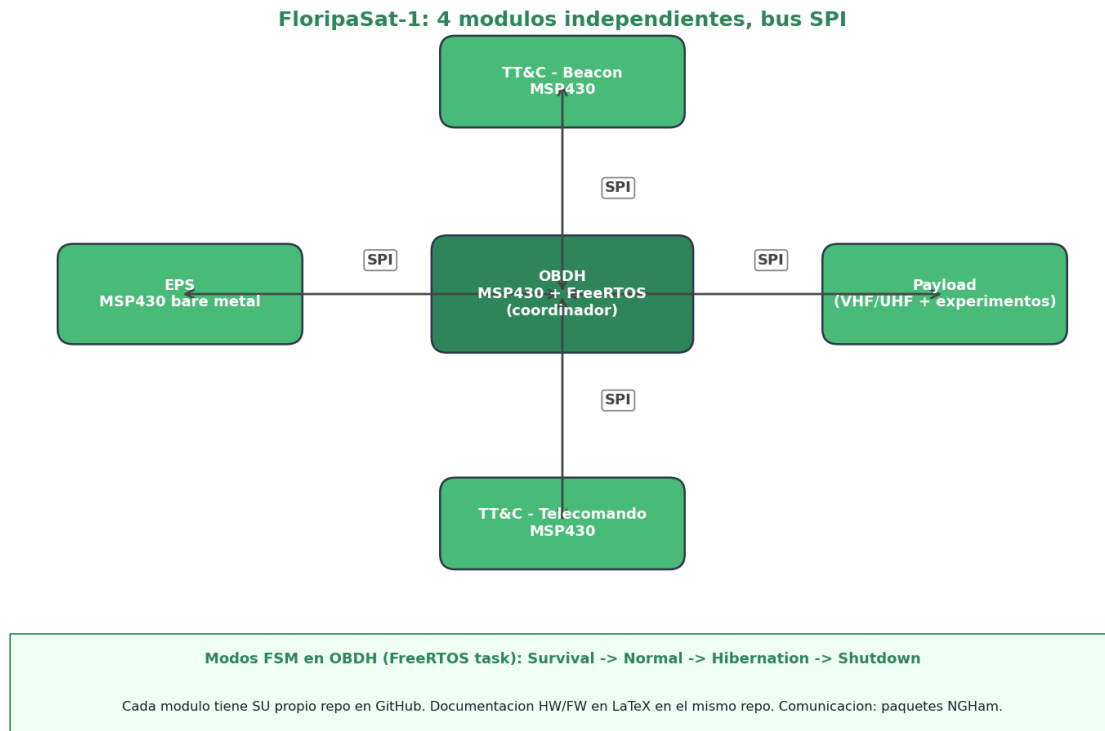


Figura 3.1: Arquitectura de FloripaSat-1: 4 módulos independientes alrededor del OBDH (On-Board Data Handling), comunicándose por SPI. Cada módulo tiene su propio MSP430 y su propio repositorio.

Detalle técnico

Por que cuatro micros y no uno solo. La razón es *tolerancia a fallas*. Si el OBDH se cuelga, el EPS sigue gestionando la energía y el Beacon sigue transmitiendo. El equipo de UFSC cita esta tolerancia como la principal razón de la arquitectura distribuida. La contra es que ahora tienes 4 firmwares que mantener en lugar de uno.

3.3. Como manejan modos / FSM

FloripaSat-1 define 4 modos operacionales que viven en una FSM dentro del OBDH (FreeRTOS task dedicada):

- **Survival.** Modo inicial post-deploy. Solo radio activa, todo lo demás apagado. Espera ventana de tierra.
- **Normal.** Operación estándar: telemetría, beacon, payload activo si la energía alcanza.
- **Hibernation.** Modo bajo consumo cuando batería baja de un umbral. Apaga payload, deja solo housekeeping mínimo.
- **Shutdown.** Crítico: batería muy baja, apaga TODO menos el watchdog que espera carga solar.

Esto se mapea casi 1-a-1 a los modos que ya definió el equipo INTISAT en el manual M12 (BOOT/SAFE/STANDBY/CAPTURA/COMM/CRITICAL). La FSM está implementada como un `switch` clásico en C, sin librerías de state machines.

3.4. Que adopta INTISAT y que no

Que adopta INTISAT

Adoptamos:

- **Repositorio por subsistema** con su propia documentacion en LaTeX. Ya lo estamos haciendo (este manual es ejemplo).
- **FSM de modos en una task dedicada** del OBDH.
- **Beacon independiente del OBDH.** Es radio amateur-friendly y nos da un canal de telemetria minima incluso si el OBDH falla.

NO adoptamos:

- **4 microcontroladores fisicos.** Nuestro presupuesto de masa y energia favorece una OBC unica (Raspberry Pi CM5) con cosubsistemas (EPS, ADCS) atendidos por micros muy chicos.
- **C bare metal.** Vamos a Python sobre Linux donde podamos porque acelera desarrollo 5-10× y nuestro equipo tiene mas experiencia ahi.

3.5. Referencias

Referencias y URLs

Web oficial. <https://floripasat.ufsc.br/>

Organizacion en GitHub. <https://github.com/floripasat>

Repositorios principales.

- `floripasat/obdh` — firmware del OBDH
- `floripasat/eps` — firmware del EPS
- `floripasat/ttc` — firmware del transceptor
- `floripasat/payloads-x` — payloads cientificos
- `floripasat/manual` — documentacion en LaTeX

Paper de referencia. Marcelino et al., “FloripaSat-1: Lessons Learned from a Brazilian Open Source CubeSat Mission”, publicado en SBESC/WSCAD/LARS (consultar el repo principal para versiones actualizadas).

Capítulo 4

OreSat (Portland State University, USA)

4.1. El proyecto en 2 paginas

Que es. OreSat es el programa de CubeSats del Portland State Aerospace Society (PSAS), de Portland State University, Oregon. Es probablemente el CubeSat universitario *mas modernamente construido* de las referencias: pensado desde el dia uno como un producto open hardware industrial-grade, con bus CANopen, OBC corriendo Linux y flight software en Python.

Misiones.

- **OreSat0** (0.5U). Lanzado el 12 de septiembre de 2022 desde Kodiak (Alaska) en el cohete Astra LV0009. La mision primaria fue demostrar el bus de cards. Operaciones limitadas por anomalia del lanzador.
- **OreSat0.5** (1U). Lanzado en marzo de 2024 en el SpaceX Transporter-10 desde Vandenberg. Misma arquitectura, forma factor mayor, ahora con experimentos cientificos.
- **OreSat-1** (3U). En desarrollo a la fecha de este manual.

Filosofia. “Cards” independientes que se conectan por bus CAN, hablando CANopen (estandar industrial CiA 301). Cada card es un STM32 con su propio Object Dictionary. La excepcion es la **C3 card**, que corre Linux con un demonio en Python llamado OLAF (*OreSat Linux App Framework*) y actua como CANopen master.

Lenguaje. Python (OLAF, Linux) para la C3. C bare metal (STM32) para el resto de cards.

4.2. La arquitectura “cards” sobre bus CAN

Detalle tecnico

Que es un Object Dictionary en CANopen. Cada nodo CANopen expone una tabla de objetos indexada por (index, subindex) de 16+8 bits. Cada objeto puede ser leido o escrito por otro nodo enviando un *SDO* (Service Data Object), o publicarse periodicamente como un *PDO* (Process Data Object). En la practica: cada card de OreSat *declara* su telemetria y sus parametros en un archivo EDS (Electronic Data Sheet), y la C3 los lee con un nombre estable sin necesidad de saber detalles del firmware de la card. Es analogo a un ROS topic, pero a nivel de hardware.

Que cards existen.

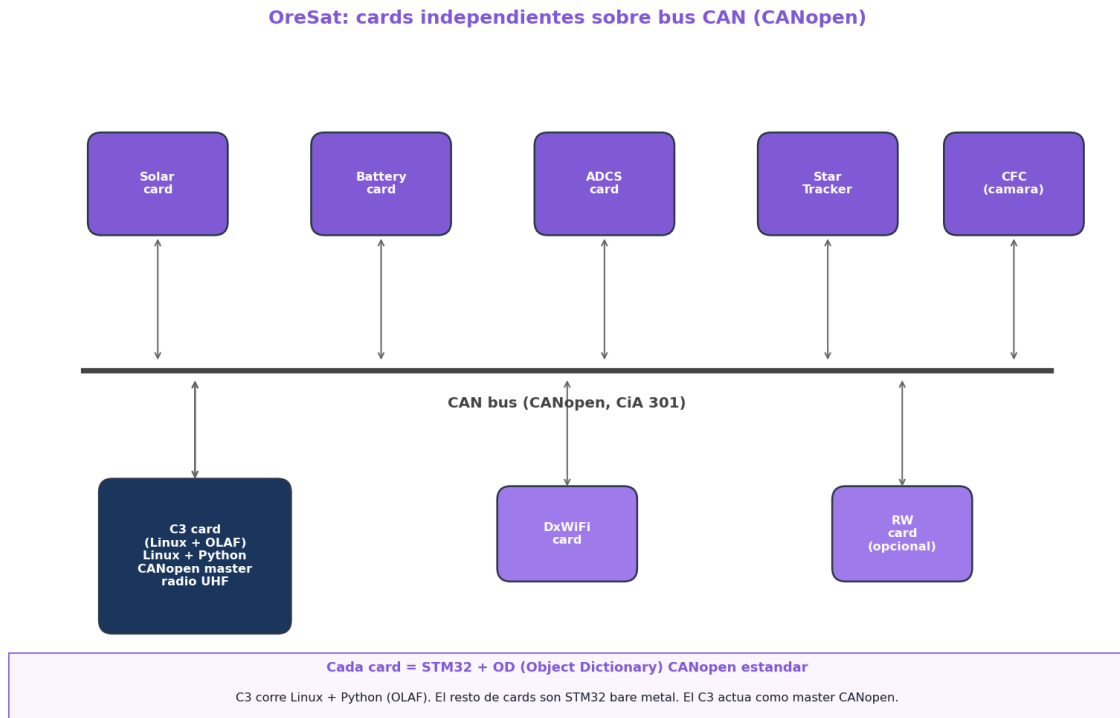


Figura 4.1: OreSat: cada subsistema es una “card” que se conecta al backplane CAN. La C3 card es el cerebro Linux. Las demas son STM32 CANopen-compliant.

- **C3 card.** Octavo Industries OSD3358-SM (TI Sitara ARM Cortex-A8 + microcontrolador PRU + DSP). Corre Debian Linux. Es el cerebro: maneja telecomando UHF, telemetria descendente, coordina las otras cards.
- **Battery card.** STM32 que gestiona el banco de baterias y reporta carga al CAN.
- **Solar card.** STM32 que mide cada panel solar y reporta corriente.
- **ADCS card.** STM32 con magnetorquers para apuntamiento.
- **Star Tracker.** OreSat publico ademas su propio star tracker hecho de una camara industrial + ARM.
- **CFC (Cirrus Flux Camera).** Camara cientifica para medir flujo de radiacion.
- **DxWiFi.** Card experimental para comunicar a alta velocidad cuando el satellite pasa sobre la estacion terrena.

4.3. Como manejan modos / FSM

La C3 card es la que mantiene la maquina de estados global. Estados documentados:

- **PRE_DEPLOY** — timer de 30 minutos post-eyeccion.
- **DEPLOY** — despliegue de antenas.
- **STANDBY** — normal, beaconing.
- **BEACON** — transmision activa al pasar sobre estacion.

- EDL — Engineering / Diagnostics Link.
- SAFE — modo de baja energia.

OLAF implementa la FSM en Python con clases explicitas y un dispatcher de eventos. Cada card tiene ademas su propia mini-FSM de salud (NMT Node State de CANopen: Initialization, Pre-Operational, Operational, Stopped) que es estandar de CANopen, no algo inventado por OreSat.

4.4. Que adopta INTISAT y que no

Que adopta INTISAT

Adoptamos:

- **Linux + Python en el OBC.** Misma eleccion: nuestro equivalente del C3 es el Raspberry Pi CM5.
- **Object Dictionary explicito (concepto).** Para INTISAT no llegamos a usar CANopen, pero la idea de *declarar* en un archivo de texto la telemetria que expone cada subsistema nos sirve para definir la API I2C de manera disciplinada.
- **Modo PRE_DEPLOY con timer obligatorio.** Es requisito del proveedor de lanzamiento para no encender transmisores antes de separarse del cohete.

NO adoptamos:

- **Bus CAN.** INTISAT 1U va a usar I2C para reducir BOM, masa y consumo. CAN seria sobre-ingenieria para 3 subsistemas.
- **Una card fisica por subsistema.** INTISAT integra OBC + payload en la misma CM5; EPS y ADCS son perifericos I2C de la CM5.

4.5. Referencias

Referencias y URLs

Web oficial. <https://www.oresat.org/>

Organizacion en GitHub. <https://github.com/oresat>

Repos clave.

- `oresat/oresat-c3` — firmware Linux + OLAF
- `oresat/oresat-olaf` — framework Python OLAF
- `oresat/oresat-configs` — Object Dictionaries de todas las cards

Standards. CANopen CiA 301 (Cabezada del CAN in Automation), disponible en <https://www.can-cia.org/can-knowledge/canopen/>.

Capítulo 5

AAUSAT (Aalborg University, Dinamarca)

5.1. El proyecto en 2 paginas

Que es. El programa AAUSAT de la Aalborg University es uno de los pioneros mundiales en CubeSats universitarios (desde el inicio de los 2000), y es probablemente el mas *metodologicamente riguroso* de la lista. Su contribucion mayor no es un satelite puntual: es un conjunto de *artefactos de ingenieria* que el resto del mundo CubeSat adopto.

Misiones.

- **AAUSAT-3** (1U). Lanzado 25 de febrero de 2013 desde Sriharikota (India) en el PSLV-C20. Demostro AIS (Automatic Identification System) marino desde orbita.
- **AAUSAT-4** (1U). Lanzado el 25 de abril de 2016 en el Soyuz Sentinel-1B.
- Tambien AAUSAT-1 (2003) y AAUSAT-II (2008) anteriores.

Filosofia. AAUSAT NO es el satelite mas elegante. Su valor esta en *como se documenta y se decide*. Tres aportes fundamentales:

1. **El documento “Operations Concept”**, que define *antes* de codear, todos los modos operacionales, sus transiciones, las ventanas de tiempo en cada modo y el comportamiento esperado del sistema.
2. **HSMM (Hierarchical State Machine Model)** — los modos no son una lista plana, sino una jerarquia donde “Normal” es padre de varios sub-modos. Esto es el patron *statechart* de Harel.
3. **CSP (Cubesat Space Protocol)**. Un protocolo de red inspirado en TCP/IP pero adaptado a links espaciales. Se libero como `libcsp` (C, MIT) y hoy lo usan decenas de misiones (incluido GomSpace, que es spin-off de la universidad).

Lenguaje. C sobre FreeRTOS para todos los modulos.

5.2. Arquitectura monolitica + CSP

A nivel hardware, AAUSAT-3/4 son monoliticos: una OBC central conectada por I2C a EPS, ADCS y radio. Lo interesante no es el cableado, es la *capa de red*: arriba de I2C corre CSP, que abstrae el bus fisico como si fuera una red IP.

Detalle tecnico

CSP en una linea. CSP es a un CubeSat lo que TCP/IP es a Internet: una capa de transporte con direcciones de nodo (5 bits), puertos (6 bits), routing por hops y soporte para encriptacion HMAC. Tu codigo NO sabe si el destino esta detras de I2C, CAN o radio: solo manda un paquete a (`node=2`, `port=10`) y CSP se encarga.

Por que importa. Te permite mover un subsistema del bus interno al downlink sin cambiar el codigo de aplicacion. Y te permite hacer testeo en tierra con un nodo CSP corriendo en una laptop como si fuera un subsistema del satelite.

5.3. Como manejan modos / FSM (HSMM)

Aqui esta la contribucion teorica fuerte. AAUSAT estructura los modos como una *jerarquia*, no una lista:

- **Detumbling** (modo padre, post-deploy)
- **Safe Mode** (modo padre)
 - Beacon ON
 - Beacon OFF (silencio total para reducir consumo)
- **Normal Mode** (modo padre)
 - Pointing
 - Payload acquisition
 - Downlink

Por que jerarquica. Las transiciones de SEGURIDAD (e.g., “bateria < 30 %”) aplican a TODOS los sub-modos del padre. Lo escribis una vez en el padre, no en cada hoja. Es exactamente la ventaja de los *statecharts* de Harel sobre las FSM planas.

5.4. Que adopta INTISAT y que no

Que adopta INTISAT**Adoptamos:**

- **Documento “Operations Concept” antes de codear.** El manual M12 cumple ese rol: define todos los modos, transiciones y comportamientos ANTES de que se escriba la primera linea de `intisat_payload_daemon.py` final.
- **Pensar los modos como jerarquia.** Aun si la implementacion en INTISAT es plana al inicio, mentalmente diferenciamos modos “operacionales” de modos “de seguridad” porque las transiciones tienen prioridades distintas.

Pendiente de evaluar:

- **Usar libcsp en INTISAT.** Es C portable, esta empaquetado para Linux y FreeRTOS, y nos daria capa de red sobre I2C. Habria que verificar si conviene la complejidad para una mision 1U.

5.5. Referencias

Referencias y URLs

Web del grupo. <https://www.space.aau.dk/cubesat/>

libcsp en GitHub. <https://github.com/libcsp/libcsp>

Documento “Operations Concept”. Buscar como “AAUSAT Operations Concept” en los reports tecnicos del grupo. El formato fue replicado por varios CubeSats europeos posteriores.

Paper canonical. “CubeSat Communication System, a New Open Source Design” — Aalborg University.

Capítulo 6

UPSat (Libre Space Foundation, Grecia)

6.1. El proyecto en 2 paginas

Que es. UPSat es el primer CubeSat en orbita que es 100 % open source: tanto el hardware (esquematicos, gerbers, BOM) como el software (firmware, ground station, protocolo de tierra). Fue desarrollado por la Universidad de Patras junto con la Libre Space Foundation (ONG sin fines de lucro con sede en Atenas) como parte de la mision QB50.

Mision.

- **UPSat** (2U). Lanzado el 18 de abril de 2017 desde Cabo Canaveral en el SpaceX CRS-11 hacia la ISS, y eyectado desde el NanoRacks deployer de la ISS poco despues. Mision: medicion de composicion de la termosfera (sensor INMS).

Filosofia. La declaracion de la Libre Space Foundation es clara: “el espacio es un commons y la unica forma de democratizarlo es publicar TODO”. UPSat es la prueba de concepto. Tambien desarrollaron la red SatNOGS (estaciones terrenas distribuidas) y herramientas como gr-satnogs (bloques GNU Radio).

Lenguaje. C sobre FreeRTOS, microcontroladores STM32.

6.2. Arquitectura

UPSat es monolitica: una OBC STM32 conectada por I2C a EPS, ADCS y radio (modulo CC1101). El payload es un INMS (Ion-Neutral Mass Spectrometer) externo. El bus interno es I2C, sin CSP ni CANopen.

Lo importante de UPSat es el PROCESO, no la arquitectura. El proyecto fue ejecutado con metodologias de ingenieria espacial reales (ECSS adaptado), con un Wiki publico donde el equipo registraba sus decisiones y problemas. Eso es la leccion mas reusable.

6.3. Como manejan modos / FSM

UPSat no documento una FSM explicita tan rica como AAUSAT. Los modos son: **DEPLOYMENT**, **NOMINAL**, **SAFE** y **EMERGENCY**, definidos en el firmware del OBC como `enum` en C. Las transiciones estan implementadas implicitamente como `if/else` dentro del main loop. Es la version mas simple posible de FSM y aun asi resultado suficiente para operar el satellite durante meses.

Intuicion

La leccion de UPSat es que **no necesitas un framework de FSM fancy para volar un CubeSat 1U/2U**. Un **switch** en C con 4 modos es suficiente. Lo que importa es:

1. Documentar los modos en un papel ANTES.
2. Que esten codificados en una sola funcion identificable.
3. Que esten testeados en tierra.

6.4. Que adopta INTISAT y que no

Que adopta INTISAT

Adoptamos:

- **Organizacion en GitHub publica con Wiki.** Es nuestro modelo de gobernanza.
- **Aceptar que 4-6 modos en una FSM plana es suficiente.** No vamos a sobre-diseñar.
- **Usar ECSS adaptado para telemetria y telecomando.** Paquetes con header de servicio (ECSS-E-ST-70-41).

NO adoptamos:

- **STM32 + C.** Nuestra OBC es Raspberry Pi CM5 + Python. UPSat representa el approach “minimal C” que NO es el nuestro.

6.5. Referencias

Referencias y URLs

Web de Libre Space Foundation. <https://libre.space/>

Organizacion GitHub. <https://gitlab.com/librespacefoundation/upsat/>
Repos.

- `upsat-obc-software` — firmware OBC
- `upsat-eps-firmware` — firmware EPS
- `upsat-ground-station-software`

SatNOGS. La red de estaciones terrenas open source que UPSat ayudo a impulsar:
<https://satnogs.org/>.

Capítulo 7

NASA cFS (Core Flight System)

7.1. Que es cFS y por que importa

cFS (Core Flight System) es un framework de software de vuelo desarrollado por NASA Goddard Space Flight Center desde fines de los 90 y liberado bajo licencia Apache 2.0 en 2015. NO es un sistema operativo: es un *conjunto de capas y aplicaciones modulares* que corren sobre un RTOS (VxWorks, RTEMS, Linux, FreeRTOS, incluso macOS para testing).

Misiones que lo usaron (parcial). LRO (Lunar Reconnaissance Orbiter, 2009), SDO (Solar Dynamics Observatory, 2010), GPM (Global Precipitation Measurement, 2014), MMS (Magnetospheric Multiscale, 2015), Lunar IceCube (2022), MAVEN, IXPE, Artemis Orion (vuelo Artemis 1 en 2022). Muchas decenas de misiones en total, incluido el primer helicoptero en Marte y misiones a Mercurio.

Por que importa para INTISAT (aun si no lo usamos).

- Es el *modelo conceptual* de como NASA piensa los modos de operacion, la telemetria, los comandos y la salud del sistema. Vale la pena entenderlo aunque escribamos otra cosa.
- Su documentacion (<https://github.com/nasa/cfs>) es de las mejores disponibles para el software de vuelo en general.

7.2. El stack en capas

De abajo hacia arriba:

- **Hardware.** Procesador embebido + perifericos.
- **RTOS/OS.** VxWorks, RTEMS, Linux, FreeRTOS.
- **PSP (Platform Support Package).** Drivers de plataforma especificos (board support package).
- **OSAL (OS Abstraction Layer).** Una API uniforme que abstrae el RTOS de abajo. Esto permite que la misma aplicacion corra sin cambios sobre VxWorks o Linux.
- **cFE (Core Flight Executive).** El runtime real: define 5 servicios fundamentales descritos abajo.
- **Aplicaciones.** Pluggable, comunican entre si por el *Software Bus* de cFE (publish/subscribe).

Los 5 servicios de cFE.

1. **ES (Executive Services).** Lanzar/parar apps, manejar tareas y memoria.

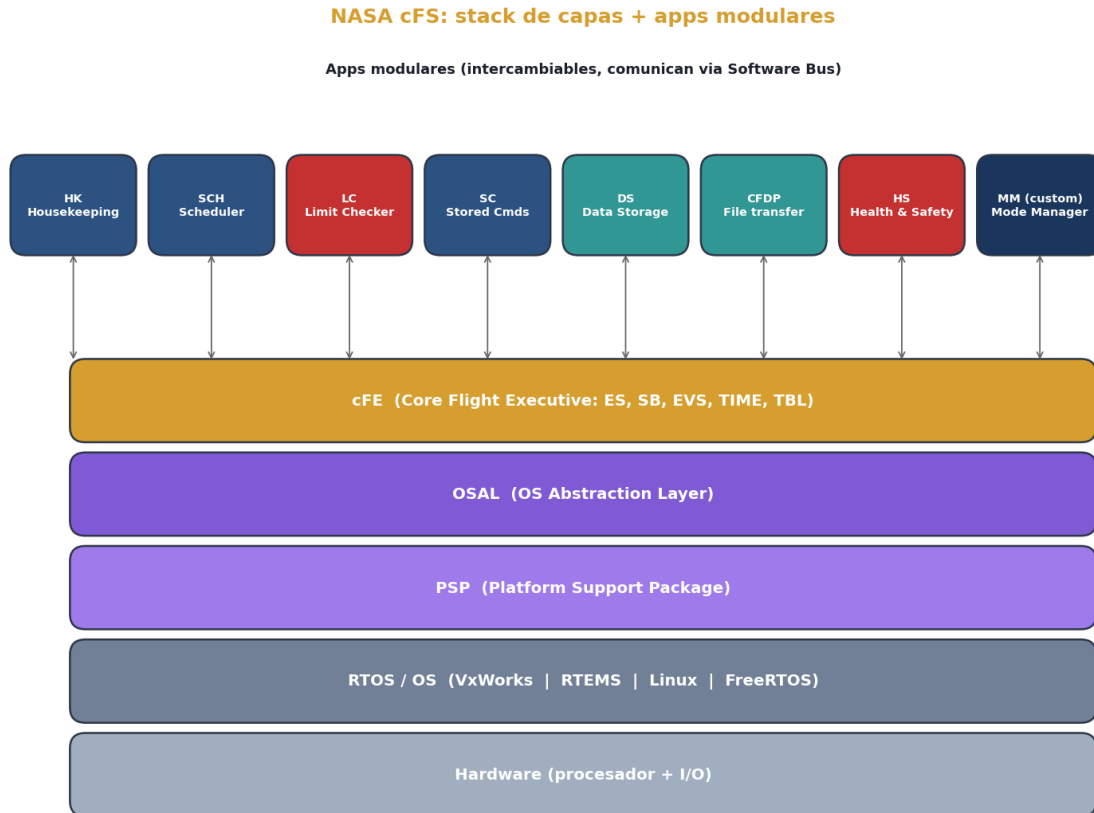


Figura 7.1: Stack de cFS: cuatro capas de plataforma (HW, OS, PSP, OSAL) sobre las que corre cFE, y arriba aplicaciones modulares intercambiables que comunican por el Software Bus (publish/subscribe).

2. **SB (Software Bus)**. Mensajería publish/subscribe entre apps. Es el bus interno del sistema.
3. **EVS (Event Services)**. Logging estructurado de eventos.
4. **TIME**. Reloj del sistema, sincronizado a tiempo de misión.
5. **TBL (Table Services)**. Tablas de parámetros actualizables desde tierra sin recompilar.

7.3. Aplicaciones modulares: el patrón clave

cFS define un *catalogo* de aplicaciones de uso común que un equipo puede tomar y meter en su misión:

- **HK (Housekeeping)**. Recolecta telemetría.
- **SCH (Scheduler)**. Lanza tareas periódicamente.
- **LC (Limit Checker)**. Vigila umbrales (voltajes, temperaturas) y dispara alarmas.
- **SC (Stored Commands)**. Cola de comandos a ejecutar cuando el satélite no está sobre tierra.
- **DS (Data Storage)**. Persistencia de telemetría.
- **CFDP**. Transferencia confiable de archivos (estándar CCSDS).
- **HS (Health & Safety)**. Watchdog y modos de seguridad.

Cada mision agrega ademas sus *aplicaciones custom*: el manejo de su payload, su MM (Mode Manager) especifico, drivers de sus sensores, etc. La clave es que TODO se comunica por el Software Bus: una app no sabe quien le esta enviando datos, solo se suscribe a un *message ID*.

Intuicion

El gran aporte conceptual de cFS para INTISAT es la separacion **Mode Manager como app aparte**. En la mayoria de los CubeSats estudiantiles, la FSM esta embebida en el “main” del firmware. cFS te obliga a pensarla como una aplicacion separada que escucha eventos y emite el modo actual al Software Bus. Esto hace mucho mas facil testearla en aislamiento.

7.4. Que adopta INTISAT y que no

Que adopta INTISAT

Adoptamos (a nivel conceptual):

- **Mode Manager como modulo separado.** En Python tendra su propio archivo (`mode_manager.py`), no embebido en el main loop.
- **Limit Checker explicito.** Un modulo que recorre la telemetria y dispara transiciones de modo cuando un umbral se cruza.
- **Stored Commands.** Una cola persistente de comandos a ejecutar.
- **Pub/sub interno (logico).** En Python lo podemos emular con un event bus simple (e.g., `asyncio.Queue` o `pyzmq`), o incluso una clase con callbacks.

NO adoptamos:

- **cFS en si.** Es C, pensado para RTOS, y portarlo a un payload Python sobre Linux es complejidad sin retorno claro para una mision 1U.

7.5. Referencias

Referencias y URLs

Repo principal. <https://github.com/nasa/cFS>

Documentacion. <https://cfs.gsfc.nasa.gov/> (con tutoriales y videos).

Apps catalog (NASA GitHub).

- `nasa/cFE` — core
- `nasa/osal` — OS Abstraction Layer
- `nasa/HK`, `nasa/LC`, `nasa/SCH`, `nasa/SC`, `nasa/DS`, `nasa/CFDP`, `nasa/HS` — aplicaciones

Tutorial recomendado. La “cFS 101” de NASA Goddard disponible en YouTube; introduce los conceptos en ~2 horas.

Capítulo 8

NASA F'(FPrime)

8.1. Que es F'y por que importa

F'(FPrime, pronunciado “F-prime”) es un framework de software de vuelo desarrollado por NASA JPL (Jet Propulsion Laboratory) y liberado bajo licencia Apache 2.0 en 2017. Es la respuesta moderna de JPL al problema que cFS resuelve: como construir software de vuelo modular y reusable. Las diferencias filosoficas con cFS son significativas.

Misiones que lo usaron.

- **ASTERIA** (CubeSat 6U, 2017–2020). El primer vehiculo en volar F'.
- **MarCO-A y MarCO-B** (mayo 2018). Primeros CubeSats interplanetarios; flyby de Marte acompanando a InSight.
- **Ingenuity** (helicoptero en Marte, 2021–2024). 72 vuelos motorizados en otro planeta; F'coordina todo desde sensores hasta motores de rotores.
- **NEA Scout** (2022). Vela solar a un asteroide.
- **LCRD** (2021). Demostracion de comunicacion laser.

Que F'haya volado en Marte es la mejor prueba de que el framework funciona en el peor escenario imaginable: una mision donde el helicoptero NO puede ser controlado en tiempo real desde la Tierra (retardo de senal de 5–20 minutos en cada sentido) y donde un bug imposibilita el vuelo.

Lenguaje. C++ (la unica de nuestras referencias).

8.2. Componentes, ports y FPP

Conceptos clave.

- **Component.** Unidad de funcionalidad. Por ejemplo, “GPS receiver”, “Camera controller”, “Mode Manager”. Cada component es una clase C++ con estado y metodos.
- **Port.** Interfaz tipada por la que dos components se comunican. Es como un metodo, pero el caller no sabe que component implementa el callee (loose coupling).
- **Topology.** El *grafo* de quien esta conectado con quien. Se describe en un archivo aparte, separado del codigo del component.
- **FPP (F Prime Prime).** El DSL textual donde describis components, ports, comandos, telemetria, eventos y *state machines*. El compilador `fpp-to-cpp` genera los stubs.

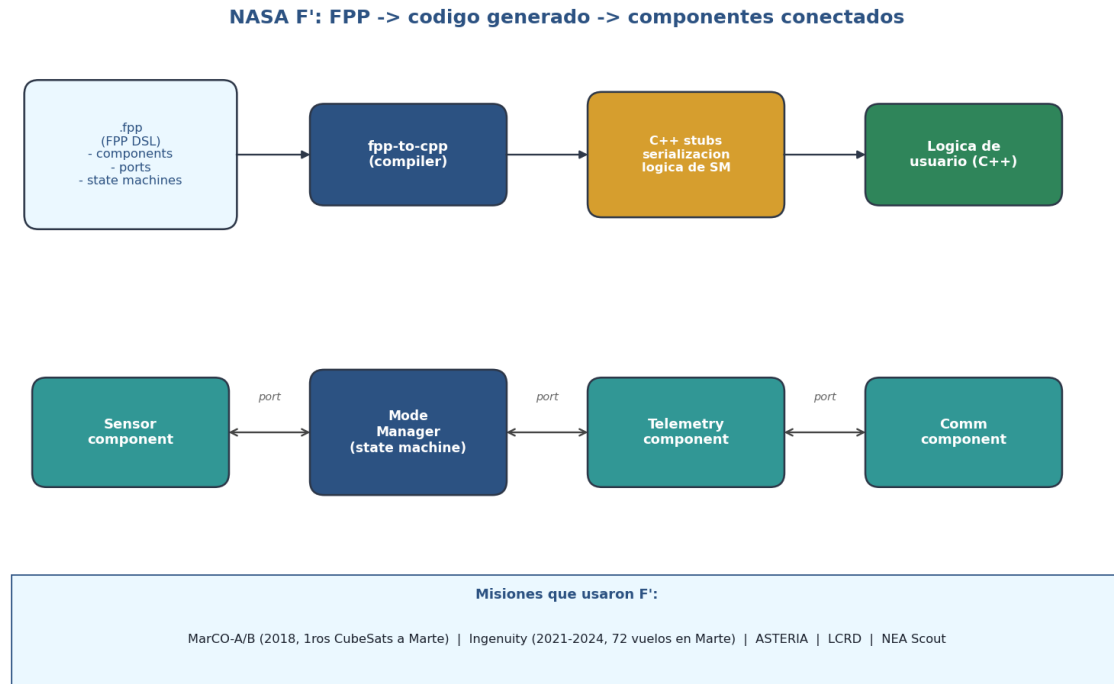


Figura 8.1: Workflow de F': el desarrollador describe su sistema en FPP (un DSL), el compilador genera stubs C++, y los componentes se conectan via *ports*.

State machines en FPP. Aquí está lo notable: en F' las FSMs se declaran en FPP y el código de las transiciones se genera. El desarrollador rellena solo las *acciones* de entrada/salida de estado. Esto reduce los errores típicos de FSM escritas a mano.

```

1 state machine SatelliteMode {
2     initial state Detumbling
3     state Detumbling
4     state Safe
5     state Normal
6     state Critical
7
8     transition Detumbling -> Safe on event RatesNominal
9     transition Safe -> Normal on event GroundCmd_Activate
10    transition Normal -> Critical on event LowBattery
11    transition Critical -> Safe on event BatteryRecovered
12 }

```

Listing 8.1: Fragmento estilizado FPP para una FSM de modos

La filosofía es opuesta a cFS: cFS usa pub/sub (acoplamiento flojo, runtime-bound), Fúsa ports tipados (acoplamiento mediano, compile-time-bound). Cada uno tiene ventajas en distintos escenarios.

8.3. Que adopta INTISAT y que no

Que adopta INTISAT

Adoptamos (conceptualmente):

- **Topología separada del código de cada componente.** Aun en Python: la conexión de módulos se describe en un archivo de configuración, no se mezcla con la lógica.

- **FSM declarativa.** El `mode_manager.py` del INTISAT declarara los estados y transiciones como datos estructurados, no como `if/else` dispersos. Esto se inspira en Fáunque sin DSL externo.

NO adoptamos:

- **F'framework completo.** C++ es ajeno al equipo y el overhead de aprender FPP no se justifica para una mision 1U.

8.4. Referencias

Referencias y URLs

Repo principal. <https://github.com/nasa/fprime>

Documentacion. <https://nasa.github.io/fprime/>

Misiones que lo usaron.

- **Ingenuity Mars Helicopter.** JPL publico un repo relacionado: <https://github.com/nasa/fprime-tutorial>
- **ASTERIA.** CubeSat de 6U operado en orbita baja terrestre entre 2017 y 2020.

FPP language reference. <https://nasa.github.io/fpp/>.

Capítulo 9

Síntesis y recomendaciones para INTISAT

9.1. Lo que NO debe pasar

Antes de las recomendaciones positivas, conviene fijar lo que queremos evitar:

Cuidado

1. **Copiar una arquitectura completa que no encaja.** Tentación: “OreSat es lindo, vamos con CAN+CANopen”. Realidad: nuestro equipo no tiene experiencia en CANopen, y para 3 subsistemas pequeños es over-engineering.
2. **Forzar el uso de un framework de NASA.** Tentación: “si Ingenuity uso F”, usemoslo”. Realidad: Fés C++ con un DSL externo (FPP) que requiere meses de aprendizaje del equipo. INTISAT no tiene ese margen.
3. **Saltearse el “Operations Concept”.** Tentación: “ya hablamos de los modos, escribimos código y vamos viendo”. Realidad: AAUSAT y NASA insisten en que el documento de operaciones se escriba ANTES del código. M12 es ese documento para INTISAT.
4. **No documentar las decisiones públicamente.** Tentación: “hagamos issues en un Slack interno”. Realidad: UPSat demostro que el Wiki público de decisiones es lo que permite que el equipo siguiente al actual entienda por que estamos haciendo lo que hacemos.

9.2. Recomendaciones de adopción selectiva

Resumen por proyecto.

1. **De FloripaSat:** repositorio por subsistema + documentación en LaTeX dentro del repo. Beacon como módulo independiente del OBDH (es nuestro futuro CubeSat radio).
2. **De OreSat:** flight software en Python sobre Linux para el OBC, declaración explícita de telemetría por subsistema (Object Dictionary conceptual), modo PRE_DEPLOY con timer obligatorio.
3. **De AAUSAT:** el documento “Operations Concept” formal (cumplido por M12), pensar modos como jerarquía (incluso si la implementación inicial es plana), evaluar libscsp para una v2 del firmware.

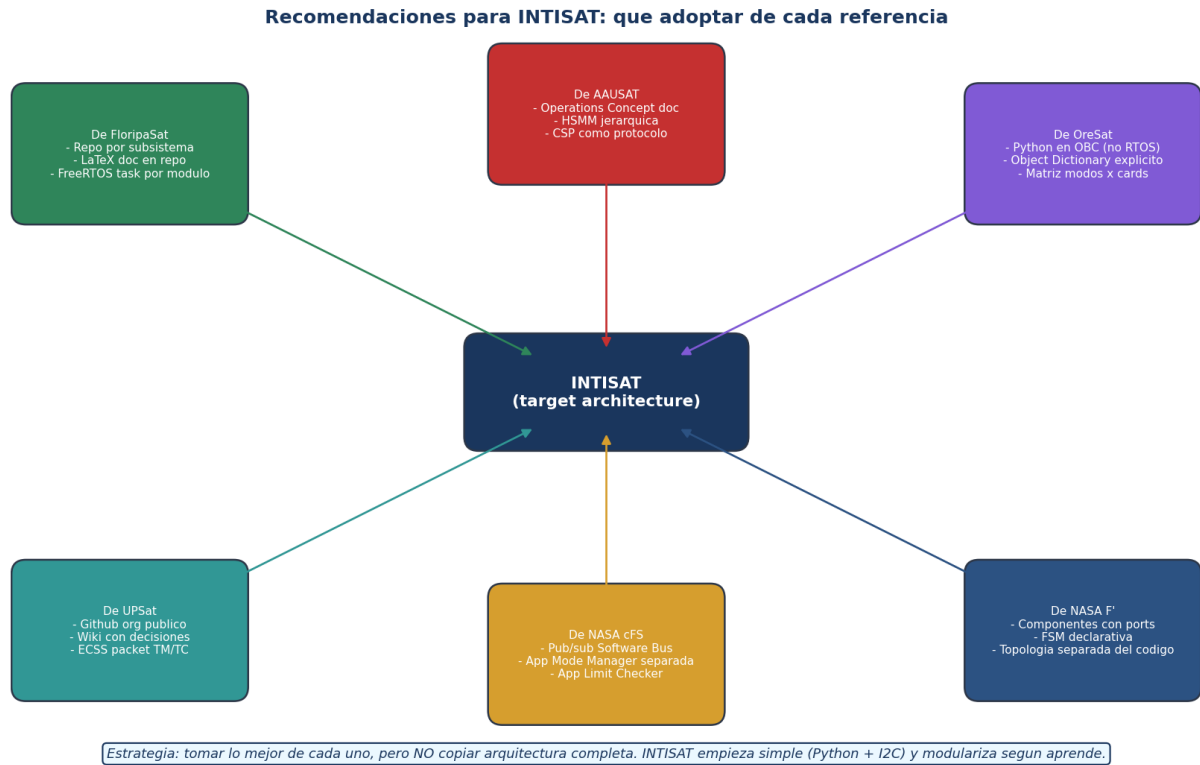


Figura 9.1: Síntesis: que toma INTISAT de cada referencia. La estrategia es *copiar inteligentemente*, no clonar.

4. **De UPSat:** github org publica con Wiki de decisiones, aceptar que 4–6 modos en FSM plana es suficiente para 1U, paquetes TM/TC con esquema ECSS adaptado.
5. **De NASA cFS:** Mode Manager como modulo separado (no embebido en el main loop), Limit Checker explícito, Stored Commands queue, comunicacion interna pub/sub.
6. **De NASA F’:** FSM declarada como datos (no `if/else` dispersos), topologia de modulos separada del código de cada modulo.

9.3. Stack final propuesto para INTISAT

Combinando todo lo anterior, el stack queda:

Detalle técnico

- **OBC:** Raspberry Pi CM5 + Linux (Raspberry Pi OS o Yocto custom).
- **Flight software:** Python 3 con `asyncio` para concurrencia. Estructura modular: `mode_manager.py`, `limit_checker.py`, `stored_cmds.py`, `telemetry_collector.py`, `payload_driver.py`.
- **Bus interno:** I2C entre OBC y subsistemas chicos (EPS, ADCS, beacon). Cada subsistema chico expone una API documentada (análogo conceptual al Object Dictionary de CANopen).
- **Telemetría / Telecomando:** paquetes binarios con header tipo ECSS (sin todo el overhead del estándar, versión simplificada). Beacon en AX.25 o FSK directo para

radio amateur.

- **Documentacion:** esta serie de manuales (M0–M13) en LaTeX dentro del mismo repositorio, con figuras generadas programáticamente desde `tools/`.
- **Modos de operacion:** segun M12 (BOOT, SAFE, STANDBY, CAPTURA, COMM, CRITICAL).

9.4. Planilla de decisiones abiertas

A pesar de toda esta informacion, hay decisiones que el equipo aun debe tomar antes de cerrar el diseno. Las listamos para que la proxima reunion las pueda abordar:

1. **¿Adoptar libcsp sobre I2C o protocolo ad-hoc?** libcsp da rigor de capa de red, pero suma una dependencia C importante. Decision en M5.
2. **¿Implementar comandos almacenados (SC) desde dia uno o como upgrade post-launch?** Riesgo: si no esta en el firmware inicial, no se puede agregar facilmente despues.
3. **¿Usar Yocto o Raspberry Pi OS oficial?** Yocto da control fino pero requiere expertise. Raspberry Pi OS arranca rapido pero tiene paquetes que no necesitamos.
4. **¿EPS firmware propio o usar uno comercial?** GomSpace y Endurosat venden EPS llave-en-mano con CSP. Costo vs tiempo de desarrollo.
5. **¿Beacon DIY o modulo comercial?** TT&C es la parte mas regulada y mas critica. Vale la pena no reinventarla.

9.5. Cierre

Construir un CubeSat no es construir un sistema nuevo: es *integrar decisiones que ya tomaron 50 equipos antes que vos*. Cada uno de los proyectos que estudiamos en este manual gasto entre 3 y 7 años de trabajo de un equipo para responder preguntas que nosotros podemos contestar leyendo su Wiki. Aprovecharlo no es debilidad: es ingeniería seria.

Lo unico que hace nuestra mision particular — el payload de FPM y la integracion con CM5 — es donde realmente vamos a aportar algo nuevo a la comunidad. Todo el resto: arquitectura, modos, telemetria, ground station, debe ser *lo mas estandar posible* para liberar tiempo a lo que si nos diferencia.

Que adopta INTISAT

Regla de oro.

1. Si una decision ya la tomaron ≥ 3 proyectos de la lista y converge a la misma respuesta, ADOPTALA sin discusion.
2. Si dividieron las respuestas mitad-mitad, ELEGI la mas simple para 1U.
3. Si nadie hizo lo que vos queres hacer, PENSA dos veces por que sos el primero. Suele significar que es mala idea o que es muy especifica de tu mision.

Apéndice A

Glosario de terminos

ADCS Attitude Determination and Control System — subsistema de actitud.

CAN / CANopen

Bus serie estandar industrial (Bosch, 1986) y su capa de aplicacion estandar CiA 301.

CCSDS

Consultative Committee for Space Data Systems. Cuerpo de estandares de comunicacion espacial.

cFE Core Flight Executive. El runtime de NASA cFS.

cFS Core Flight System. Framework de software de vuelo de NASA Goddard.

CSP Cubesat Space Protocol. Protocolo de red para CubeSats creado en AAUSAT.

ECSS European Cooperation for Space Standardization. Cuerpo de estandares europeos.

EDS Electronic Data Sheet. Archivo de declaracion de Object Dictionary en CANopen.

FPP F Prime Prime. DSL textual de NASA F'.

FreeRTOS

RTOS open source, el mas popular en CubeSats.

FSM Finite State Machine. Maquina de estados finita.

HSMM

Hierarchical State Machine Model. FSM con jerarquia de modos (statecharts de Harel).

I2C Inter-Integrated Circuit. Bus serie 2-hilos de Philips.

NMT Network Management. Servicio CANopen para coordinar estado de nodos.

OBDH / OBC

On-Board Data Handling / On-Board Computer. El cerebro del satelite.

OLAF OreSat Linux App Framework. Framework Python de OreSat.

OSAL OS Abstraction Layer. Capa de portabilidad de RTOS.

PDO Process Data Object (CANopen). Publicacion periodica de telemetria.

PSP Platform Support Package (cFS). Drivers de plataforma.

RTOS Real-Time Operating System.

SatNOGS

Red distribuida open-source de estaciones terrenas de Libre Space Foundation.

SDO Service Data Object (CANopen). Lectura/escritura sincronica de objetos.

SPI Serial Peripheral Interface. Bus serie 4-hilos.

TM/TC

Telemetry / Telecommand.

TT&C Telemetry, Tracking and Command. Subsistema de comunicaciones.

Apéndice B

Índice de URLs verificables

B.1. Misiones universitarias

- FloripaSat: <https://floripasat.ufsc.br/>
- FloripaSat GitHub: <https://github.com/floripasat>
- OreSat: <https://www.oresat.org/>
- OreSat GitHub: <https://github.com/oresat>
- AAUSAT: <https://www.space.aau.dk/cubesat/>
- UPSat (Libre Space): <https://libre.space/projects/upsat/>

B.2. Frameworks NASA

- NASA cFS GitHub: <https://github.com/nasa/cFS>
- NASA cFS docs: <https://cfs.gsfc.nasa.gov/>
- NASA F'GitHub: <https://github.com/nasa/fprime>
- NASA F'docs: <https://nasa.github.io/fprime/>
- FPP language: <https://nasa.github.io/fpp/>

B.3. Protocolos y estándares

- libcsp: <https://github.com/libcsp/libcsp>
- CANopen CiA 301: <https://www.can-cia.org/can-knowledge/canopen/>
- CCSDS estándares: <https://public.ccsds.org/Publications/>
- ECSS estándares: <https://ecss.nl/>

B.4. Infraestructura terrestre open

- SatNOGS: <https://satnogs.org/>
- Libre Space Foundation: <https://libre.space/>
- GNU Radio: <https://www.gnuradio.org/>